AI : Report

TD1-WARGAME

Written by : Waneeza Hira AWAN

Group : 1

M1 SIC, 2026-2027.

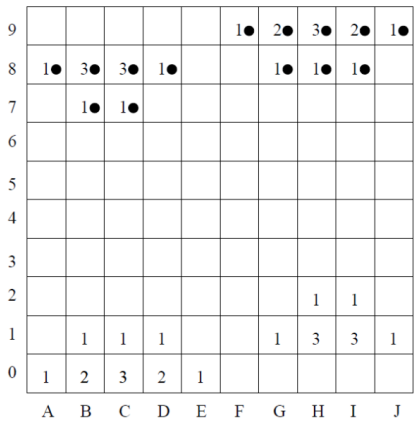


Table des matières

1. Game Mechanics and Successor Generation 2
2. Implementing the Minimax Decision Logic 3

1. Game Mechanics and Successor Generation

To begin this assignment, I first had to understand how this simplified wargame works. The game is played on a 10x10 board with two players. The objective is simple: either capture all enemy pieces, or reach the opponent's back rank with at least one piece to win right away. Each turn, a player can move only one piece by one square in any of the 8 directions. When attacking an enemy piece, combat is resolved by comparing the total strength of surrounding friendly pieces versus enemy pieces across the 8 neighboring squares. Before coding the decision-making logic, my priority was to give the computer a way to find all possible moves for any board state.

In the template code, there was no function to list valid moves. Since an AI needs to test every option during its turn, I created a simple Coup struct containing (l1, c1) for the starting square and (l2, c2) for the destination. Storing only these coordinates is lightweight and avoids duplicating entire 100-cell boards needlessly. To collect all available actions, I used a singly linked list (ListeCoups). Because the number of possible moves changes constantly during a match, a linked list lets me allocate just what is needed. I also added ajouter_coup to insert new moves at the front and liberer_coups to free the allocated memory after checking the moves.

The generation itself happens in generer_coups. It loops over the whole board to find pieces belonging to the current player. For every piece found, it checks the 8 neighboring squares using relative offsets. Instead of rewriting all the movement rules, I called f_test_mouvement, which was already provided. This function handles boundary checks and prevents moving onto an ally's square. If the move is valid, it gets added to the list. With this done, the AI has its candidate moves ready for the Minimax search tree.

2. Implementing the Minimax Decision Logic

After having a move generator, my next task was to code the core decision-making loop using the Minimax concept. Since this is a turn-based, two-player game, the idea is fairly simple: my AI wants to pick the move that leads to the highest possible score, while keeping in mind that the opponent will always try to pick the move that hurts my position the most. To make the code readable and directly follow what we saw in class, I decided to split this logic into two mutually recursive functions: f_max for my turns and f_min for the opponent's turns.

When writing these functions, my main concern was how to handle the board copies without slowing down the program or causing memory leaks. At first, allocating dynamic memory with malloc or f_raz_plateau inside every branch seemed like an easy way out, but it quickly became messy and inefficient with thousands of calls. Instead, I chose to allocate a local board array directly on the stack (Pion copie[...]) inside the loop. For every legal move, I duplicate the current state using f_copie_plateau, execute the action with f_bouge_piece, and recursively call the opposite function with profondeur - 1 and -joueur to flip the turn. Once we hit profondeur == 0 or if no moves are available, the state is evaluated from my AI's point of view via f_eval. Finally, I make sure to call liberer_coups at the end of each function to free the linked list before returning the best score.